

Please note that the challenge would be revised on a real-time basis – so keep checking the document frequently

Adobe India Hackathon 2025 – Finale

Connecting the Dots Challenge

Theme: From Brains to Experience – Make It Real

Introduction

Congratulations on making it to the Finale!

You've already built the brains — a robust PDF understanding engine (Round 1A) and a persona driven- document intelligence system (Round 1B).

Now it's time to bring those brains to life in a real user experience.

In this round, you'll design and present an interactive, intelligent PDF reading application that shows the world how your engine works — and why it matters.

Your Mission

Build a web-based reading experience powered by your earlier work that:

1. Displays PDFs beautifully using Adobe's PDF Embed API (<https://developer.adobe.com/document-services/docs/overview/pdf-embed-api/>).
2. Connects the dots by showing related sections from the same or other documents.
3. Helps users explore & understand content faster with context aware-recommendations.
4. Adds an "Insights" feature to deliver knowledge beyond the page.

Core Features

Your application should:

- Users should be able to upload their PDFs to the app in bulk to represent documents they've read in the past.
- Users should be able to open a fresh PDF to represent the document they are opening for the first time.
- Display the document: "Must render PDFs with 100% fidelity and support zoom/pan interactions.
- Highlight related sections: "Must identify and highlight at least 3 relevant sections with >80% accuracy.

Short snippet explanations: "Snippets should be 1-2 sentences and clearly explain relevance".

Let users jump to related sections with a single click.

- Make the experience fast and responsive — no long waits for recommendations. Navigation should complete in <2 seconds

Follow-On Features

• Insights Bulb:

Use an LLM (GPT-4o) to provide:

- o Key insights
- o "Did you know?" facts
- o Contradictions / counterpoints
- o Inspirations or connections across docs

Podcast Mode:

Generate a short (2–5 min) narrated audio overview based on:

- o Current section
- o Related content found by your system
- o Insights from the Bulb feature

Constraints

- Base app (recommendations) must run on CPU, ≤ 10 sec response time.
- Follow-on features can call an LLM (GPT-4o).
- UI must be usable in Chrome.

Deliverables

- Working prototype (link or demo environment).
- Private GitHub repository with:
 - o Frontend + backend code
 - o README with setup instructions
 - o Notes on how to run without internet for base features
- Pitch deck (max 5–6 slides) covering:
 - o Problem & vision
 - o Your solution & key features
 - o Tech innovation & key innovation
 - o Demo flow
 - o Optional features (if add)

Optional: Short demo video (≤ 2 min) for the Jury.

Evaluation Parameters:

LLM Environment & Evaluation Details

During the evaluation, your solution will be provided with credentials to access the Gemini-2.5-Flash model from Gemini. Candidates must ensure that API keys are not hardcoded in their code.

We will run your solution using a command similar to:

```
docker run -e LLM_PROVIDER= -e OPENAI_API_KEY= -e OPENAI_API_BASE= -e  
OPENAI_MODEL= -p 8080:8080 your-solution-image
```

Examples

Using Gemini:

```
docker run -e LLM_PROVIDER=gemini -e  
GOOGLE_APPLICATION_CREDENTIALS=<PATH_TO_CREDS> -e  
GEMINI_MODEL=gemini-2.5-flash -p 8080:8080 your-solution-image
```

Using Azure OpenAI:

```
docker run -e LLM_PROVIDER=azure -e AZURE_OPENAI_KEY=<AZURE_API_KEY> -e  
AZURE_OPENAI_BASE=<AZURE_API_BASE> -e  
AZURE_API_VERSION=<AZURE_API_VERSION> -e  
AZURE_DEPLOYMENT_NAME=gpt-4o -p 8080:8080 your-solution-image
```

Using OpenAI directly:

```
docker run -e LLM_PROVIDER=openai -e OPENAI_API_KEY= -e  
OPENAI_MODEL=gpt-4o -p 8080:8080 your-solution-image
```

Using Ollama (candidates may use for local development):

```
docker run -e LLM_PROVIDER=ollama -e OLLAMA_BASE_URL=http://localhost:11434  
-e OLLAMA_MODEL=llama3 -p 8080:8080 your-solution-image
```

Please use the sample script (https://github.com/rbabbar-adobe/sample-repo/blob/main/chat_with_llm.py) provided to make LLM calls in a way that conforms to the environment variable standards in the script.

For non-Python solutions, you may implement equivalent logic in your language of choice, if it uses the same environment variables.

Text-to-Speech (TTS) Library Requirement

During the evaluation process, your solution will be provided access to Azure's Text-to-Speech service via the following environment variables: TTS_PROVIDER (with a value of 'azure'), AZURE_TTS_KEY, and AZURE_TTS_ENDPOINT.

For development purposes, candidates may use Google's Text-to-Speech API or any local solution of their choice. However, the implementation must adhere to the usage of the above-mentioned environment variables for compatibility, as these will be passed to your solution during evaluation.

Please refer to the provided sample script (https://github.com/rbabbar-adobe/sample-repo/blob/main/generate_audio.py) for guidance on converting text to audio files in accordance with the expected environment variable structure.

If you are using a non-Python language, you may replicate the functionality in your language of choice, as long as it respects the same environment variable conventions.

How should your solution build

Your git repo should have the code for both frontend and backend, and a Dockerfile, and should build with the below command.

```
docker build --platform linux/amd64 -t yourimageidentifier .
```

How should your solution run

Your solution should run with the below command

```
docker run -e LLM_PROVIDER=gemini -e  
GOOGLE_APPLICATION_CREDENTIALS=<PATH_TO_CREDS> -e  
GEMINI_MODEL=gemini-2.5-flash -e TTS_PROVIDER=azure -e  
AZURE_TTS_KEY=TTS_KEY -e AZURE_TTS_ENDPOINT=TTS_ENDPOINT -p  
8080:8080 yourimageidentifier
```

This should bring up a web application which should be accessible on <http://localhost:8080/>

Pro Tips for the Finale

Tell a story — show how a real persona benefits from your app.

Rehearse your demo — make it tight and predictable.

Highlight the magic moments — your best recommendation, your smartest insight.

Prepare for questions — think about performance, scalability, and user adoption.

Quantify your impact: Prepare metrics showing time saved or comprehension improved

Key Dates

Finale pitch day: 5th September 2025 TBD

Pitch deck submission deadline: 3rd September 2025 TBD

Code freeze: 19th August 2025 TBD